

Code-Layout, Readability and Reusability

Date	1 July 2026
Team ID	—
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	5 Marks

This document evaluates the quality of the Rising Waters codebase in terms of its structure, readability, and reusability.

Code Layout Checklist

S. No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	PEP8-style 4-space indentation across app.py and notebooks
2	Proper File Structure	Files and folders are logically organized	Yes	Separated into model/, data/, notebooks/, static/, templates/
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g. FEATURE_NAMES, MODEL_ACCURACY, X_train_scaled
4	Function / Method Names	Functions are descriptively named	Yes	cap_outliers_iqr(), predict_form(), predict()
5	Code Comments	Inline and block comments explain logic	Yes	Docstring header in app.py; numbered section comments in training script
6	Modular Design	Code is split into reusable functions/modules	Partial	Training pipeline is modular; app.py could split routes into a blueprint as it grows
7	No Redundant Code	Duplicate or unused code is removed	Yes	FEATURE_CONFIG dictionary is reused across GET/POST/error branches
8	Error Handling	Exceptions and errors are handled gracefully	Yes	try/except in /predict re-renders the form with an error message

Reusable Components / Modules

S. No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	FEATURE_CONFIG dictionary	Python	GET form render, POST prediction, error re-render in app.py	High
2	cap_outliers_iqr()	Python (pandas)	Applied to every numeric feature column during preprocessing	High
3	scaler.pkl (StandardScaler)	scikit-learn	Both offline evaluation and live Flask prediction use the same fitted scaler	High
4	style.css	CSS	Shared across home.html, predict.html, flood_result.html, no_flood_result.html	High
5	metadata.json loader	Python/JSON	Drives model name & accuracy display and feature ordering	Medium

Overall Code Quality Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clear separation of concerns across folders; consistent style
Readability	4	Descriptive names and docstrings make intent easy to follow
Reusability	4	Shared config/scaler pattern avoids duplication between training and serving
Documentation / Comments	4	README and inline comments cover setup, data, and pipeline steps well
Overall Score	4	Solid, maintainable codebase for the project's scope